AnalystBot v6 재구축 지시서 — 단일 스키마 + 구조화 출력 + 재생 테스트

Oct 6, 2026 · @jung youngjin

0. 지위·범위·원칙

지위 — 이 문서는 **이전 지시서 전부를 대체한다**: v5 본문, v5-A 부록, 게이트2·오픈월드·서치 재구축·인도네시아–태국·페이블 해석 접목·북아일랜드–조지아 작업 지시서. 이전 문서와 충돌하면 이 문서만 본다. 이전 문서의 **내용**(검증 규칙, 판정 수학, 필수형, 상한, 해석 규칙 목록)은 §6 "가져갈 것"에 적힌 만큼만 살아 있다. `docs/fable/inbox/2026-10-06_v6_rebuild.md`로 넣고, 이전 지시서들은 `docs/fable/archive/`로 옮긴다.

범위(사용자 결정 그대로) — 봇의 일은 하나: **사용자가 묻는 경기를, 물었을 때, 제대로 분석해 카드를 낸다.** 자동 배포·매일 슬레이트·채점·결과 원장·정답지·metrics는 범위 밖이다. 비용 원장과 예산 가드는 지출 관리라 유지한다. 유료 API는 Solar(검색 결과 읽기·변수 추출)와 DeepSeek(가설·시나리오·반증) 둘뿐이다.

원칙 다섯

1. **단계 경계는 타입 하나** — 단계 사이에 dict·tuple·문자열 덩어리가 다니지 않는다. `app/core/schema.py`의 pydantic 모델만 다닌다.
2. **LLM 출력은 구조화 출력으로 강제** — 산문을 파싱하지 않는다. 출력 모양은 schema.py의 모델에서 생성한 JSON 스키마로 API가 강제한다.
3. **한 가지 일은 한 곳에서** — 문지기·사다리·상한·날짜 저장·LLM 호출이 각각 정확히 하나. 사본이 생기면 빌드가 실패한다.
4. **새로 짓고, 통과하면 갈아끼우고, 옛것은 지운다** — 옛 코드를 고치지 않는다. `app/core/` 옆에 짓고, §7 재생 테스트가 통과하면 `cmd_analyze`를 새 경로로 바꾸고, 옛 경로를 삭제한다.
5. **숫자는 실측만** — 상한·추론 강도·문턱은 녹화본이나 실측으로 정한다. 이 문서의 숫자 중 실측이 아닌 것은 "초기값"으로 표시돼 있고, §8의 실측으로 덮어쓴다.

1. 왜 다시 짓는가

10-04~10-06 결함을 종류별로 묶으면 넷이고, 뿌리는 둘이다.

종류	실제 사례	뿌리
① 꼴 불일치	Game을 dict로 앎, `dict(v)`에 Classified, 반증 튜플을 dict로 읽어 카드 사망, NPB players dict, redteam이 Draft에 dict 섞음, favors `opponent`를 판정층이 모름	단계 경계 타입이 한 곳에 없다

종류	실제 사례	뿌리
② 모델이 모르는 칸	direction_hint 자유 문장, 카탈로그 이름 미전달, 팀 이름 대신 열쇠, url 머리표, 산문 속 JSON, done 미호출, 추론이 출력 상한을 다 먹고 content 0자	LLM 출력을 산문 파싱 — 구조화 출력 미사용
③ 끊긴 배선·죽은 코드·사본	spend_out·hypo.run 미호출, 날짜가 세 경로에서 버려짐, 문지기 둘(querygate·query_ok), cap_for 사본, 사다리 둘, 세포·에이전트 루프·하위 추출 경로 공존	v3→v4→v5를 덧붙이기만 하고 지우지 않음
④ 테스트가 조각만 봄	계약 2,096개, 돌연변이가 반복 통과 (M89·M93·M104·M110·M125·M126·M128·M131·M132·M133), 카드 전체를 보는 테스트 없음	경로 전체를 재생하는 테스트가 없다

①②③은 모양이 다를 뿐 같은 병이다 — 고치면 옛 경로나 사본이 다른 모양을 기대해서 다음이 터진다. 이음매가 수백 개라 땜질로는 끝나지 않는다. v6는 이음매 자체를 없앤다: 타입 하나(①), 구조화 출력(②), 단일 경로와 삭제(③), 재생 테스트(④).

2. v6 구조

파일 배치 — 새 코드는 전부 `app/core/` 아래. 기존 부품은 import해서 쓰되, 기존 부품이 dict를 돌려주면 `app/core/adapt.py`의 변환 함수 **한 곳**에서 schema 타입으로 바꾼다(부품 내부는 건드리지 않는다).

```
app/core/
  schema.py     # 단계 경계 타입 전부 (§3)
  llm.py        # LLM 호출 함수 하나 (§4)
  adapt.py      # 기존 부품 출력 → schema 타입 변환, 여기서만
  pipeline.py   # 일곱 단계를 순서대로 부르는 함수 analyze(question) -> Card
  stages/
    s1_resolve.py   s2_facts.py   s3_docs.py   s4_extract.py
    s5_verify.py    s6_hypo.py    s7_judge.py
  render.py     # Card -> 텍스트 카드
tests/e2e/      # 재생 테스트 (§7)
```

일곱 단계 흐름 (각 단계는 앞 단계 출력 타입만 받는다)

1. **s1 경기 확정** `Question → MatchRef` — 이름표·열쇠·오픈월드 ⑥단·36시간 창. 후보가 둘 이상이면 `Ambiguous`를 돌려 사용자에게 묻는다.
2. **s2 구조 사실** `MatchRef → Facts` — 어댑터(fotmob·ESPN·statsapi 등)와 경기 페이지에서 순위·최근 결과·상대전·XI·대회 형식.
3. **s3 문서 수집** `MatchRef, Facts → list[Doc]` — 하위 과제 8개(유형군 4 × 팀) 질의를 코드가 만들고, 검색→거르기→재정렬→fetch→htmldate. Doc 하나에 본문·발행일·출처가 한 몸으로 붙는다.

4. **s4 변수 추출** `list[Doc], Facts → list[VarCandidate]` — 하위 과제마다 Solar 구조화 출력 **1회**. 출력 모양은 스키마가 강제한다.
5. **s5 검증** `list[VarCandidate], list[Doc] → list[Variable], list[Drop]` — 인용이 그 Doc 본문에 글자 그대로 있는가, 날짜, 신원, 방향 재계산(subject/favors), 중복 병합, 해석 규칙 변수 추가.
6. **s6 가설·시나리오** `list[Variable], Facts → HypoSet` — DeepSeek 단계 3(가설·시나리오)과 단계 4(반증), 코드 필수형 보강.
7. **s7 판정** `HypoSet, Board | None, Facts → Card` — 보드 대입, 생존율, 상한, 방향 문장, 등급, 방아쇠.

상태 하나 — 실행 중 누적되는 것(비용, 장부, 깔때기, 폐기 사유)은 `RunState` 객체 하나에 담아 각 단계에 넘긴다. 전역 변수·모듈 캐시에 실행 상태를 두지 않는다(병렬·재생에서 새는 원인이었다).

3. schema.py 명세

모든 모델은 pydantic v2 `BaseModel`, `model_config = ConfigDict(extra="forbid", frozen=True)`. 열거값은 `Literal` 또는 `Enum` — 자유 문자열 금지. 아래는 필수 칸이다. 칸을 더하는 건 되지만 빼거나 이름을 바꾸려면 questions/에 적고 사용자 결정을 받는다.

모델	칸	비고
`TeamKey`	`str` 패턴 `^team:\d{6}$`	이름이 아니라 열쇠. 이름은 렌더 때만
`MatchRef`	id, sport(Literal), competition, competition_short, home: TeamKey, away: TeamKey, kickoff_utc(datetime, tz 필수), venue, langs: list[str], openworld: bool	competition_short는 §2-2 함수 결과
`XI`	team: TeamKey, state(Literal confirmed/predicted/none), formation: str|None, players: list[str], bench: list[str], source_url	어댑터별 모양 차이는 adapt.py에서 흡수
`Facts`	standings, recent(팀별 최근 N경기 스코어 목록), h2h, xi: dict[TeamKey, XI], match_format(Literal league/group/knockout_single/two_leg_1st/two_leg_2nd/final_single/friendly/unknown), extra_time: bool|None, sources: list[str]	없으면 None, 지어내지 않음
`Doc`	id, url, domain, tier(Literal), lang, title, body, published_at(datetime|None), date_source(Literal meta/jsonld/time_tag/engine/none), subtask: SubtaskId	**본문과 날짜가 한 몸** — 따로 저장하는 경로 없음

모델	칸	비고
`SubtaskId`	Literal 8개(A-home…D-away)	
`VarCandidate`	subtask, type(카탈로그 Literal), subject: TeamKey, claim, quote, doc_id	**LLM 출력 모델** — 방향 칸 없음(방향은 코드가 정한다)
`Variable`	id("R1"…), type, subject, favors(Literal home/away/both/goals_up/goals_down), hardness(Literal hard/soft), weight(float), claim, quote, doc_ids: list, from_rule: str|None, usable_as_basis: bool	favors·weight는 코드가 계산
`Drop`	candidate 요약, reason(사유 Enum), detail	버린 이유가 반드시 남는다
`Hypothesis`	team: Literal home/away, reasons: list[HypoLine], breaks: list[HypoLine]	`HypoLine` = text + var_ids
`Scenario`	id(A…H), outcomes: list[tuple[int,int]], driver_vars: list[str], mechanism: str, type(Literal), added_by(Literal deepseek/code/redteam), strength(float)	
`HypoSet`	hypotheses(2), scenarios, redteam: list[RedTeamLine], drops	
`Board`	lines: list[BoardLine]	BoardLine = market(Literal), side, point|None — 배당 칸 없음
`MarketEval`	line: BoardLine, marks: dict[str, Literal○/△/✗], survival: float, cap: float, grade(Literal 낸다/소액/소액하단/없음), dead: list[str]	dead에는 ✗와 △ 모두
`Card`	match, facts 요약, variables, drops 요약, hypo, evals, direction: str, conclusion: str, triggers: list[str], funnel, cost, started: bool	render.py의 유일한 입력

LLM 출력 모델은 따로 표시 — `VarCandidate`, DeepSeek용 `HypoOut`·`RedTeamOut`. 이 셋만 llm.py가 스키마로 바꿔 보낸다. LLM 출력 모델에는 코드가 정할 칸(favors·weight·strength·grade)을 **두지 않는다** — 모델이 그 칸을 채울 길 자체를 없앤다.

4. llm.py 명세 — 호출 함수 하나

```
def ask(*, provider: Literal["solar","deepseek"], purpose: str,
        messages: list[Message], out: type[T], state: RunState) -> T |
LlmFail
```

앱 어디서든 LLM은 이 함수로만 부른다. `openai` SDK·`httpx`로 LLM을 직접 부르는 코드가 app/에 남아 있으면 빌드 실패(§7 정적 검사).

Solar — 구조화 출력(strict)

- `response_format={"type":"json_schema","json_schema":{"name":..., "strict":true, "schema":...}}`. Upstage 문서 규칙대로 스키마를 만든다: 모든 object에 `additionalProperties: false`, 모든 속성을 `required`에 넣고, 선택 칸은 타입에 `null`을 더해 표현한다. pydantic의 `model_json_schema()` 결과를 이 규칙으로 바꾸는 변환 함수를 llm.py 안에 하나 둔다.
- **경기마다 달라지는 enum은 요청마다 스키마에 박는다** — `subject`는 그 경기 두 팀 열쇠만 허용하는 enum, `doc_id`는 그 하위 과제에 넘긴 문서 id만 허용하는 enum, `type`은 그 하위 과제 담당 카탈로그 유형만. 모델이 다른 값을 낼 길이 없다. 이게 "이름 대신 열쇠", "머리표 대신 도메인", "자유 문장 방향" 같은 결함을 구조적으로 막는 자리다.
- 추론 강도: 추출(`purpose="extract"`)은 **minimal**(10-06 실측: low는 출력 4,000을 추론에 다 쓰고 content 0자, minimal은 추론 0·content 3,519자). 다른 용도는 녹화본으로 실측해 정한다. 값은 `config/llm.yaml`의 purpose별 표에서 읽는다.
- `finish_reason == "length"`이면 실패로 처리(사유 `llm_truncated`) — 잘린 JSON을 고치려 하지 않는다.

DeepSeek — 공식 문서를 읽어 지원 모드를 확인하고 결과를 config에 적는다. JSON 스키마 강제를 지원하면 Solar와 같은 방식, JSON 모드만 지원하면 `json_object` + 프롬프트에 스키마 첨부 + pydantic 검증. 어느 쪽이든 반환은 `out` 타입이다.

검증·재시도 — 응답을 `out.model_validate_json()`으로 검증. 실패하면 검증 오류 문장을 붙여 **1회만** 재요청. 두 번째도 실패하면 `LlmFail(reason="llm_schema")`. 타임아웃·5xx·429·truncated·schema는 서로 다른 사유다.

녹화·재생·원장 — 모든 호출은 카세트(기존 §5 구현 재사용)를 거친다. 원장 한 줄: provider, purpose, model, reasoning_effort, in/cache/out/reasoning 토큰, USD, finish_reason, 소요 초. 예산 판정은 `RunState`의 지갑 하나로.

5. 단계별 명세

각 단계는 `stages/sN_*.py`의 함수 하나 `run(input..., state: RunState) -> 출력`. 단계 안에서 실패해도 예외를 위로 던지지 않고, 빈 출력 + `state.funnel`에 사유를 남긴다. 다음 단계는 빈 입력을 받아도 돈다(카드는 항상 나온다).

s1 경기 확정 — 기존 이름표·열쇠 사다리·오픈월드 ⑥단·`window.inside`를 adapt.py로 감싸 `MatchRef`를 만든다. 오픈월드 ⑥-1의 철자 후보 질문도 `llm.ask(purpose="spell")`로 바꾼다. 확정 기준(두 팀 일치·36시간 창·정확히 1개)은 그대로.

s2 구조 사실 — 기존 어댑터 호출 → adapt.py에서 `Facts`로. 어댑터마다 다른 모양(players dict/list 등)은 여기서만 흡수. `match_format`은 대회명·라운드 정보로 정하고 모르면 unknown.

s3 문서 수집 — 하위 과제 8개를 병렬로. 하위 하나:

1. 질의 2개를 코드가 만든다: `{team} {opp} {word}`(영어) + 같은 꼴 현지어. `{word}`는 search_words.yaml의 개념 낱말(유형 이름 금지), `{team}`은 영어 짧은 표기, 어깻점 등 문지기 금지 문자는 값에서 제거. 템플릿 변수는 team/opp/comp/coach/player/word뿐.
2. 문지기 **하나**(llm.query_ok와 querygate 중 하나만 남긴다 — §6)를 통과한 질의만 SearXNG(time_range=week), 결과 5건 미만이면 Bing RSS. GDELT는 경기당 상한 안에서.
3. 도메인 등급 → 신원 선필터 → 날짜 힌트 → 재정렬 → 상위 3건 fetch → htmldate → `Doc`. 발행일이 킥오프 7일 밖이면 Doc을 만들지 않는다(사유 stale).
4. 같은 URL은 경기 단위 캐시에서 한 번만 받고, 캐시는 `Doc` 통째로 저장한다(본문만 저장하는 경로 없음).

s4 변수 추출 — 하위마다 `llm.ask(provider="solar", purpose="extract", out=list[VarCandidate])` 1회. 입력은 그 하위의 Doc 문단(BM25+재정렬 상위)과 머리표 `[doc_id]`. 스키마가 subject·doc_id·type을 enum으로 강제(§4). 결과 0개이고 하위 검색 예산이 남으면, 1차 결과 제목에서 코드가 고유명사 2개를 뽑아 좁힌 질의로 s3①~③을 한 번 더 → 추출 1회 더. 그 이상은 없다.

s5 검증 — 기존 검증 규칙의 **내용**을 새 함수로 옮긴다: 인용이 해당 Doc 본문에 글자 그대로(NFKC·합자표·따옴표 정규화), 날짜(Doc.published_at), 신원, 방향 재계산(subject+type→favors, catalog 규칙), 매체 2곳 이상이면 hard, MinHash 병합, 표본 부족은 `usable_as_basis=False`. 해석 규칙(IR01~IR12)이 Facts에서 변수를 더한다(규칙 문서의 표 그대로, 사실 없으면 건너뜀). weight는 config/weights.yaml에서.

s6 가설·시나리오 — DeepSeek 단계 3: 입력은 `usable_as_basis=True` 변수만 근거 목록으로, 나머지는 context_only로. 출력 `HypoOut`(가설 문장+var_ids, 시나리오 outcomes·driver_vars·mechanism). 코드 검사: driver·var_ids가 근거 목록 밖이면 그 줄 버림(사유 cited_excluded/unknown_var). 필수형 보강(실점동반·대량득점·회귀·난타전·90분 무승부·해석 규칙 requires_scenario). 단계 4 반증: 추가만, 생존율 상승 금지. strength = 유형별 최댓값 합(weights).

s7 판정 — 보드가 있으면 보드 줄만, 없으면 기준선 복수. 다중 스코어 시나리오는 스코어 전부 대입(○/△/✗). 생존율, 상한(XI·오픈월드·자료 얇음), 같은 방향 3개 상한, 시작한 경기 등급 없음, 블록 9 방향 문장(차이 0.15 이상이면 팀 이름, 미만이면 반반+기울기), 방아쇠. 이 단계는 LLM을 부르지 않는다.

6. 가져갈 것·지울 것

가져간다 — 부품으로 import, 내부는 건드리지 않음 (출력은 adapt.py에서 타입으로)

부품	쓰는 단계
어댑터(fotmob HTML·ESPN·statsapi·NPB·KBO·fdorg)	s2
팀 열쇠 발급기·별칭표·이름표 사다리·오픈월드 ⑥단 ·window(inside/same_match_time)	s1
검색 통로(SearXNG·GDELT·Bing RSS)·도메인 등급표 ·robots·신원 선필터·datehint·rerank(전용 스레드 소유)·htmldate·trafilatura	s3
search_words.yaml 개념 낱말, competition_short, Babel 언어	s3
검증 규칙 내용(인용 정규화·NFKC·합자표·MinHash·출처 등급)	s5로 **옮겨 씀**
판정 수학(생존율·상한 표·필수형 조건·같은 방향 3개 ·verdict 사다리)	s7로 **옮겨 씀**
해석 규칙 IR01~IR12 표, weights.yaml	s5·s6
카세트(녹화·재생·replay_miss·fallthrough), 비용 원장	llm.py·전 단계

"옮겨 씀"은 규칙의 **내용**을 새 타입 위에서 다시 쓴다는 뜻이다. 옛 함수를 import해서 dict를 넘기지 않는다.

지운다 — 갈아끼운 뒤(§8 4단계) 삭제 커밋

- 세포: 검색 계획기·세포 장부 경로 전부 (장부는 RunState 칸으로 대체)
- 에이전트 루프(agentloop)·done 도구·```json 블록 추출·산문 JSON 파서·done 거부 ·DONE_RESERVE
- 하위 과제 에이전트 경로·subtasks의 옛 run_subtask(새 s3·s4로 대체)
- 문지기 둘 중 하나: `llm.query_ok`와 `querygate.gate`를 비교해 하나만 남긴다(남길 쪽 기준: 오픈월드 ⑥-2와 s3 질의 모두 통과시키는 쪽). 남긴 쪽 이름으로 모든 호출을 바꾼다
- 사본 전부: cap_for 사본, 두 번째 verdict 사다리, 날짜 저장 함수 여럿(store_doc 계열은 s3 캐시 하나로)
- direction_hint 처리, 프롬프트에 enum 목록 적는 코드(스키마가 강제하므로 불필요)
- 범위 밖: 슬레이트·스케줄러·launchd·발송·채점·결과 원장·정답지(golden_v5)·metrics/ ·stage_gates.yaml·gate2_report

- 옛 경로만 지키던 계약 테스트 — 삭제 전에 목록과 "v6에서 이 보장은 어디로 갔나"(스키마/llm.py/E2E 불변식/단위)를 한 줄씩 적은 표를 `docs/fable/archive/v5_contracts_map.md` 로 남긴다

7. 테스트 체계

핵심 — E2E 재생 테스트 (`tests/e2e/`)

녹화본으로 `pipeline.analyze()` 전체를 오프라인으로 돌려 `Card` 를 검사한다. LLM·검색 호출 0회, 비용 0, 결과 결정적. 경기 5개:

경기	녹화본	확인 포인트
온두라스-자메이카	hon_jam_2 (있음)	오픈월드 입구, 경우의 수 형식(group), 자메이카 대량 결장
북아일랜드-조지아	새로 녹화	어깻점 감독명(O'Neill), 양 팀 변수 균형
포르투갈-노르웨이	새로 녹화(경기 종료 후라도 입력은 같음)	자팀 결장·8강 확정의 방향(상대 쪽)
인도네시아-태국	새로 녹화	5백·에이스 벤치·9-2 보정·단판 결승 → 저득점·90분 무승부 필수
KBO 1경기(당일)	새로 녹화	야구 경로, 선발 상대전 규칙(IR08), 불펜(IR09)

카드 불변식 — 다섯 경기 전부에 같은 검사

1. 카드가 나온다(예외 없음), 블록 1~9 전부 존재, 방향 문장이 팀 이름 또는 "반반"
2. 모든 Variable의 quote가 doc_ids 중 하나의 본문에 정규화 후 글자 그대로 있다
3. 모든 Variable의 subject가 그 경기 두 팀 열쇠 중 하나, favors가 enum 안
4. 자팀 결장 변수의 favors가 자팀 쪽이 아니다 / 확정 팀의 stakes가 그 팀 쪽이 아니다
5. 가설·시나리오가 인용한 var_id가 전부 usable_as_basis=True
6. 조건 걸린 필수형 시나리오가 전부 존재
7. ✗ 또는 △ 시나리오가 있는 마켓의 survival < 1.0, XI 상한 적용, started면 grade 없음
8. 반증이 시나리오를 지우지 않았다, 어떤 마켓도 반증 후 survival이 오르지 않았다
9. Doc마다 published_at 또는 date_source=none 사유가 있고, date_source=none인 Doc을 근거로 한 변수가 없다
10. 깔때기 합계가 맞는다(받은 후보 수 = 통과 + 버림 사유 합)

불변식 외에 카드 텍스트 스냅샷(syrupy 등)을 남겨 의도치 않은 변화가 diff로 보이게 한다. 스냅샷 갱신은 사유를 커밋 메시지에 적을 때만.

단위 테스트 — 수학과 파서만: s7 판정 수학(생존율·상한·다중 스코어 대입·방향 문턱), s5 인용 정규화, 해석 규칙 각각(고정 Facts 입력), competition_short, llm.py의 스키마 변환(additionalProperties·required·null). 돌연변이 검사는 s7 판정 수학과 s5 인용 대조에만 건다.

정적 검사 — 빌드 실패 조건

- pyright strict: `app/core/` 전체 에러 0
- `app/` 어디서든 LLM SDK·HTTP 직접 호출이 llm.py 밖에 있으면 실패(AST)
- `app/core/`의 단계 함수 서명에 `dict` · `Any` · `tuple` 반환이 있으면 실패
- 문지기 함수가 둘 이상 정의돼 있으면 실패

속도 — 전체 테스트(E2E 5경기 재생 포함) 1분 이내. 테스트 중 실모델·실API 호출 금지(재생만).

8. 이행 순서 — 옆에 짓고, 갈아끼우고, 지운다

각 단계 끝에 그 단계의 테스트가 초록이어야 다음으로 간다. 옛 코드(`app/` 기존 경로)는 4단계 전까지 **수정하지 않는다** — 옛 봇은 그대로 돌 수 있어야 한다.

단계	할 일	합격	상한
1	schema.py 전부 + llm.py(스키마 변환·Solar strict·DeepSeek 모드 확인·검증·1회 재시도·카세트·원장) + adapt.py 골격	pyright strict 0, llm.py 단위 테스트, **Solar strict 실호출 1회**로 enum 강제 확인(일부러 잘못된 값을 유도하는 프롬프트에도 스키마 밖 값이 안 나오는지)	반나절
2	s1·s2·s3 (adapt로 기존 부품 감싸기)	다섯 경기 각각 s3까지 라이브 1회 → Doc 목록 녹화. Doc마다 발행일 또는 사유, 같은 URL 중복 0	반나절
3	s4·s5·s6·s7·render·pipeline	다섯 경기 각각 v6 전체를 라이브 1회(녹화 켬) → 이 녹화본이 E2E 고정물. 카드 불변식 10개 전부 통과	하루
4	`cmd_analyze`를 `pipeline.analyze`로 교체. 같은 질문으로 옛 봇·새 봇 카드를 나란히 1경기 보고	E2E 5경기 + 단위 + 정적 검사 초록	1시간
5	§6 "지운다" 목록 삭제 + v5_contracts_map.md	삭제 후 E2E·단위·정적 검사 초록, `app/`에 옛 경로 import 0	반나절

녹화본에 대해 — v5 녹화본(hon_jam_1·2)은 LLM 요청 모양이 달라 v6 재생에 맞지 않는다. 검색·fetch 부분만 재사용 가능하면 쓰고(fallthrough로 확인), LLM 부분은 3단계 첫 라이브로 새로 녹화한다. 경기당 라이브 비용은 10-06 실측 $0.06~0.24 범위, 5경기 합 하루 상한 안.

3단계에서 수치 정하기 — 추출 1회의 실제 출력 토큰·추론 토큰을 원장으로 보고, 하위 출력 상한을 "실측 최대 × 1.5"로 정해 config에 적는다. 다른 상한(경기 출력, fetch 수)도 같은 방식. 이 문서에 적힌 상한 숫자는 그때 덮어쓴다.

막혔을 때 — 같은 단계에서 같은 꼴의 실패가 두 번 나오면 고치기 전에 멈추고, 그 실패가 schema·llm.py·단일 경로 원칙 중 무엇으로 막혔어야 했는지 한 줄 적어 보고한다. 원칙으로 못 막는 종류면 이 문서를 고칠 일이다 — 혼자 우회하지 않는다.

9. 금지·보고·마감

금지

- 4단계 전에 옛 경로(`app/` 기존 코드) 수정 — 고치고 싶은 결함이 보이면 v6에서 그 자리를 새로 짓는다
- 단계 경계에 dict·tuple·Any, LLM 출력 산문 파싱, llm.py 밖 LLM 호출, 문지기·사다리·상한의 두 번째 정의
- LLM 출력 모델에 코드가 정할 칸(favors·weight·strength·grade) 두기
- 상한을 올려서 잘림 해결 — 잘리면 추론 강도·입력 크기를 먼저 실측으로 조정
- 실측 없이 숫자 정하기, 이 문서에 없는 새 기능, 리그를 leagues.yaml에 넣어 우회
- 테스트 중 실모델·실API, 글자 검사 테스트, git stash, 치환 후 diff 확인 생략
- 하루 비용 상한 변경(사용자 결정 사항)

보고 — 단계마다 한 번

```
§8 단계 n 완료 — 테스트 x/x · pyright 0 · 커밋 abc1234 · 라이브 n회 $0.00 · 소요 h시간
막힌 것: (없음 | 한 줄)
```

3단계는 다섯 경기 카드 전문 + 불변식 10개 표(경기×불변식 ✓/✗). 4단계는 옛 봇·새 봇 카드 나란히. 5단계는 삭제한 파일 목록과 줄 수, v5_contracts_map.md 경로. 수치는 카드·원장 원문에서 복사한다 — 로그를 따로 세서 쓰지 않는다.

마감 조건 — 5단계까지 초록이면 끝이다. 그 뒤 코드 변경은 사용자가 실제 카드의 오류를 가져올 때만, 그 오류를 재현하는 녹화본을 E2E에 추가한 뒤에 고친다. 그 전에 "개선"은 하지 않는다.